LABORATORY WORK №6

Topic: SQL JOINs

Objective: To understand and practice different types of SQL JOIN operations and learn the difference between ON and WHERE clauses.

Part 1. Database Setup

1.1. Create Tables

sql

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    salary DECIMAL(10,2)
);

CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50),
    location VARCHAR(50)
);

CREATE TABLE projects (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(50),
    dept_id INT,
    budget DECIMAL(10,2)
);
```

1.2. Insert Sample Data

sql

```
INSERT INTO employees VALUES
(1, 'John Smith', 101, 50000),
(2, 'Jane Doe', 102, 60000),
(3, 'Mike Johnson', 101, 55000),
(4, 'Sarah Williams', 103, 65000),
(5, 'Tom Brown', NULL, 45000);

INSERT INTO departments VALUES
(101, 'IT', 'Building A'),
(102, 'HR', 'Building B'),
(103, 'Finance', 'Building C'),
(104, 'Marketing', 'Building D');

INSERT INTO projects VALUES
(1, 'Website Redesign', 101, 100000),
(2, 'Employee Training', 102, 50000),
(3, 'Budget Analysis', 103, 75000),
(4, 'Cloud Migration', 101, 150000),
(5, 'AI Research', NULL, 200000);
```

Part 2. CROSS JOIN

2.1. Basic CROSS JOIN

sql

```
SELECT e.emp_name, d.dept_name
FROM employees e
```

```
CROSS JOIN departments d;
```

Answer: The result contains 20 rows (5 × 4 = 20).

2.2. Alternative CROSS JOIN Syntax

sql

```
-- a) Comma notation
SELECT e.emp_name, d.dept_name
FROM employees e, departments d;

-- b) INNER JOIN with TRUE
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON TRUE;
```

2.3. Practical CROSS JOIN

sql

```
SELECT e.emp_name, p.project_name
FROM employees e
CROSS JOIN projects p;
```

Part 3. INNER JOIN

3.1. Basic INNER JOIN with ON

sql

```
SELECT e.emp_name, d.dept_name, d.location
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Answer: 4 rows are returned. Tom Brown is not included because his dept_id is NULL.

3.2. INNER JOIN with USING

sql

```
SELECT emp_name, dept_name, location
FROM employees
INNER JOIN departments USING (dept_id);
```

Answer: USING displays the common dept_id column only once.

3.3. NATURAL INNER JOIN

sql

```
SELECT emp_name, dept_name, location
FROM employees
NATURAL INNER JOIN departments;
```

3.4. Multi-table INNER JOIN

sql

```
SELECT e.emp_name, d.dept_name, p.project_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id
INNER JOIN projects p ON d.dept_id = p.dept_id;
```

Part 4. LEFT JOIN

4.1. Basic LEFT JOIN

sql

```
SELECT e.emp_name, e.dept_id AS emp_dept,
    d.dept_id AS dept_dept, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

Answer: Tom Brown is included, but his department information is NULL.

4.2. LEFT JOIN with USING

sql

```
SELECT emp_name, dept_id, dept_name
```

```
FROM employees
LEFT JOIN departments USING (dept_id);
```

4.3. Find Unmatched Records

```
sql
SELECT e.emp_name, e.dept_id
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

4.4. LEFT JOIN with Aggregation

```
sql
SELECT d.dept_name, COUNT(e.emp_id) AS employee_count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY employee_count DESC;
```

Part 5. RIGHT JOIN

5.1. Basic RIGHT JOIN

```
sql
SELECT e.emp_name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

5.2. Convert to LEFT JOIN

```
sql
SELECT e.emp_name, d.dept_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

5.3. Find Departments Without Employees

sql

```
SELECT d.dept_name, d.location
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

Part 6. FULL JOIN

6.1. Basic FULL JOIN

sql

```
SELECT e.emp_name, e.dept_id AS emp_dept,
    d.dept_id AS dept_dept, d.dept_name
FROM employees e
FULL JOIN departments d ON e.dept_id = d.dept_id;
```

Answer: Left-side NULL values represent departments without employees. Right-side NULL values represent employees without departments.

6.2. FULL JOIN with Projects

sql

```
SELECT d.dept_name, p.project_name, p.budget
FROM departments d
FULL JOIN projects p ON d.dept_id = p.dept_id;
```

6.3. Find Orphaned Records

sql

```
SELECT
  CASE
    WHEN e.emp_id IS NULL THEN 'Department without employees'
    WHEN d.dept_id IS NULL THEN 'Employee without department'
```

```
        ELSE 'Matched'
    END AS record_status,
    e.emp_name,
    d.dept_name
FROM employees e
FULL JOIN departments d ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL OR d.dept_id IS NULL;
```

Part 7. ON vs WHERE Clause

7.1. Filtering in ON Clause

sql

```
SELECT e.emp_name, d.dept_name, e.salary
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id
AND d.location = 'Building A';
```

7.2. Filtering in WHERE Clause

sql

```
SELECT e.emp_name, d.dept_name, e.salary
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.location = 'Building A';
```

Answer: ON keeps all employees, while WHERE filters out employees whose departments are not in Building A.

7.3. ON vs WHERE with INNER JOIN

sql

```
-- ON
```

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id
AND d.location = 'Building A';

-- WHERE
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id
WHERE d.location = 'Building A';
```

Answer: Both queries return the same results because INNER JOIN only returns matching rows.

Part 8. Complex JOIN Scenarios

8.1. Multiple Joins with Different Types

sql

```
SELECT d.dept_name, e.emp_name, e.salary,
       p.project_name, p.budget
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
LEFT JOIN projects p ON d.dept_id = p.dept_id
ORDER BY d.dept_name, e.emp_name;
```

8.2. Self JOIN

sql

```
ALTER TABLE employees ADD COLUMN manager_id INT;

UPDATE employees SET manager_id = 3 WHERE emp_id = 1;
```

```
UPDATE employees SET manager_id = 3 WHERE emp_id = 2;

UPDATE employees SET manager_id = NULL WHERE emp_id = 3;

UPDATE employees SET manager_id = 3 WHERE emp_id = 4;

UPDATE employees SET manager_id = 3 WHERE emp_id = 5;


SELECT e.emp_name AS employee,
    m.emp_name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

8.3. Join with Subquery

sql

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM departments d
INNER JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 50000;
```

Part 9. Lab Questions

9.1. What is the difference between INNER JOIN and LEFT JOIN?

INNER JOIN returns only matching rows. LEFT JOIN returns all rows from the left table.

9.2. When would you use CROSS JOIN in a practical scenario?

CROSS JOIN is used to generate all possible combinations of two tables.

9.3. Why does the position of a filter condition (ON vs WHERE) matter for outer joins but not for inner joins?

ON filters matching rows before the outer join is completed. WHERE filters the final result. INNER JOIN returns the same result in both cases.

9.4. What is the result of CROSS JOIN if table1 has 5 rows and table2 has 10 rows?

5 × 10 = 50 rows.

9.5. How does NATURAL JOIN determine which columns to join on?

It automatically joins columns with the same names in both tables.

9.6. What are the potential risks of using NATURAL JOIN?

It may join unexpected columns and produce incorrect results.

9.7. Convert LEFT JOIN to RIGHT JOIN.

sql

```
SELECT *
FROM B
RIGHT JOIN A ON A.id = B.id;
```

9.8. When should you use FULL OUTER JOIN?

When you need all records from both tables, including unmatched records.

Part 10. Additional Challenges

10.1. Simulate FULL OUTER JOIN using UNION

sql

```
SELECT *
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id

UNION

SELECT *
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

10.2. Find Employees Working in Departments with More Than One Project

sql

```
SELECT DISTINCT e.emp_name, d.dept_name
```

```
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
JOIN projects p ON d.dept_id = p.dept_id
GROUP BY e.emp_id, e.emp_name, d.dept_name
HAVING COUNT(p.project_id) > 1;
```

10.3. Organizational Structure Using Self-Joins

```
sql
SELECT
    e.emp_name AS employee,
    m.emp_name AS manager,
    mm.emp_name AS top_manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id
LEFT JOIN employees mm ON m.manager_id = mm.emp_id;
```

10.4. Find All Pairs of Employees in the Same Department

```
sql
SELECT e1.emp_name AS employee1,
       e2.emp_name AS employee2,
       e1.dept_id
FROM employees e1
JOIN employees e2
ON e1.dept_id = e2.dept_id
AND e1.emp_id < e2.emp_id;
```

Conclusion

In this laboratory work, I practiced different types of SQL JOIN operations, including CROSS JOIN, INNER JOIN, LEFT JOIN, RIGHT JOIN and FULL JOIN. I also learned how to use ON, USING and

NATURAL JOIN clauses. I understood the differences between ON and WHERE and practiced combining multiple tables using SQL queries.